Unit-1

- Introduction to DevOps
- Key Principles & Practices
- Benefits of DevOps
- Collaboration & Communication
- Customer Centric Decision making

Assignment-1 : Please see in last slide.

Dev Ops

Agile methodology and DevOps are two complementary practices that bring efficiency and predictability to all aspects of modern day software development. They are not the same, but they work together to make software development faster, smoother, and more reliable.

- As we saw , the agile methodology is an iterative software development approach that focuses on collaboration, rapid software releases, and customer feedback. Agile is a cultural and management philosophy that aims to get every team member to focus on continuous improvement and value delivery to customers. In simplest way , Agile is like the **game plan** for how teams should work and adapt quickly.
- DevOps is a software delivery approach that automate and streamline how code goes from development to the user / operations teams. DevOps teams use tools and practices to automate processes that historically have been manual and slow—such as deploying code or provisioning infrastructure. In simplest way , DevOps is like the **transport vehicle** making sure the code ships smoothly and efficiently to the user.

How Agile and DevOps Work Together ? Get that clarity …

- Agile helps teams decide what to build and build it iteratively.
- DevOps helps them test, deploy, and deliver it quickly and automatically.

Agile developers provide software increments that DevOps integrates and deploys through continuous delivery and automation.

Dev Ops (contd)

- DevOps is a set of practices, tools, and cultural principles that combine software development and IT operations to automate and integrate the processes between the two teams.
- It aims to shorten the development lifecycle and provide continuous delivery of high-quality software by improving collaboration, communication, and automation.
- It eliminates the traditional separation between development (Dev) and operations (Ops) teams and foster a shared sense of responsibility for the product for Customer !!
- Technologies like containers (e.g., Docker) and orchestration tools (e.g., Kubernetes) help make it easier to manage applications throughout their lifecycle

Key Components of DevOps Lifecycle / Essentials of DevOps pipeline

The DevOps lifecycle consists of seven phases representing the processes, capabilities, and tools needed for development

Continuous Development –

It focuses mostly on project planning and coding. At this phase, stakeholders and project needs are gathered and discussed. In addition, the product backlog is maintained based on customer feedback and is divided down into smaller releases and milestones to facilitate continuous software development.

Dev Ops (contd)

Continuous Integration –

Continuous integration is the most important stage of the DevOps lifecycle. At this phase, updated code or new functionality and features are developed and incorporated into the existing code. Also , the code is tested before each commit. In addition, the necessary tests are planned during this period.

Continuous Testing –

Some teams conduct the continuous testing phase prior to integration, whereas others conduct it after integration. In the event of a bug or error, the code is returned to the integration phase for correction. automation testing minimises the time and effort required to get reliable findings. During this stage, teams use technologies like as Selenium, BrowserStack etc

Continuous Deployment –

This is the most important and active step of the DevOps lifecycle, during which the finished code is released to production servers in an automated and repeatable way. This is done with the help of many deployment tools like Kubernet , Terraform etc. The tools usage ensure consistency throughout the development, testing, production, and staging environments.

Continuous Feedback –

Constant feedback is implemented to assess and enhance the development phase. During this phase, client behavior is examined for each release in an effort

Dev Ops (contd)

future releases and deployments. Companies can collect feedback using either a structured or unstructured ways.

Continuous Monitoring –
During this phase, the functioning and features of the application are regularly monitored to detect system faults such as low memory, security vulnerabilities or a non-reachable server etc. This procedure enables the IT staff to swiftly detect product performance issues and their underlying causes.

Continuous Operations –
The final phase of the DevOps lifecycle is essential for minimising scheduled maintenance and other planned downtime. Typically, developers are forced to take the server offline in order to perform updates, which increases the downtime and could cost the organisation a large amount of money.

Agile Vs Dev Ops

Core Distinction:

Aspect	Agile	DevOps
Continuous Development	Refers to frequent code changes during short iterations (sprints). Code is evolved rapidly based on feedback from stakeholders.	Refers to the automation of moving new code from commit to deployable artifact. It ensures every code change can move smoothly into production (with minimal friction).
Continuous Integration (CI)	Focus is on team collaboration and committing changes often to a shared repository. Testing is usually manual or semi-automated.	Focus is on automation: Every code commit triggers automated builds and test suites to ensure quality and readiness for delivery.
Continuous Testing	May involve manual testing, user stories validation, and QA teams working in parallel with developers.	Automated testing (unit, integration, UI, security) is tightly integrated into the CI/CD pipeline. Tests are run as code progresses, ensuring defects are caught early and releases remain fast.
Feedback	Internal (from stakeholders) during sprint	External (from production systems): performance metrics,

Phase – Continuous Development

Continuous Development –

It focuses mostly on project planning and coding. At this phase, stakeholders and project needs are gathered and discussed. In addition, the product backlog is maintained based on customer feedback and is divided down into smaller releases and milestones to facilitate continuous software development.

- DevOps starts with clarity of intent. Every failure that occurs later in testing, deployment, or production can almost always be traced back to poor thinking during the planning phase.
- In this phase, the goal is not to build software, but to build understanding. Teams must clearly understand who the user is, what problem the user is facing, and why that problem is worth solving.
- Continuous Development does not mean continuously writing code. It means continuously refining what needs to be built. This phase assumes by default that requirements are incomplete at the beginning and will evolve over time.
- In this phase it requires collaboration between all stakeholders involved in the system. This includes end users, developers, and those who will eventually operate and maintain the system.

1. Identifying the Core User Need

The first responsibility of a DevOps team in the planning phase is to identify the core user need. This is not the same as listing features. A user need describes a problem or pain point experienced by a specific group of users in a specific context. is worth solving.

Phase- Continuous Development (contd)

2. Refining the Need into Actionable Objectives

Once the core user need is identified, it must be refined into actionable objectives..

Objectives describe what the system should enable the user to do, without yet prescribing how it will be implemented

3. Breaking Objectives into Smaller Development Units

After objectives are defined, they must be broken down into smaller development units that can be worked on incrementally. These units are not tasks or code modules yet; they are logical pieces of functionality that can be delivered, reviewed, and improved independently.

4. Explicitly Considering Feedback Mechanisms

One of the most important characteristics of Continuous Development is the deliberate inclusion of feedback mechanisms. Feedback is not an afterthought; it is designed into the system from the planning stage itself. Teams must ask how they will know whether a feature is useful, confusing, or unnecessary.

Phase – Continuous Integration

Continuous Integration –

- *Continuous integration is the second most important stage of the DevOps lifecycle. At this phase, updated code or new functionality and features are developed and incorporated into the existing code. Also , the code is tested before each commit. In addition, the necessary tests are planned during this period.*
- Modern software is never built by a single person working in isolation. It is built incrementally, by multiple contributors, often in parallel, and frequently under changing requirements.
- At the end of CD phase, teams shall have a well-defined backlog containing multiple small development units that are expected to evolve over time. At this point, planning clarity exists, but no value has yet reached users.
- The question that is addressed is not "how do we safely combine everyone's work without breaking the system?".
- Continuous Integration shifts the mindset from "my code works" to "our system works." This shift is fundamental because DevOps optimizes for system-level reliability rather than individual productivity.

1. Frequent Merging

Developers are expected to merge their work regularly rather than accumulating large changes over time. This practice reduces the scope of each integration and makes failures easier to diagnose.

Phase- Continuous Integration (contd)

2. Automated Validation

Every integration triggers a set of checks that verify whether the system can still be built and whether core functionality remains intact. These checks are not only meant to prove correctness in all scenarios, but to catch obvious breakages also early.

3. Rapid Feedbacks

The third activity is rapid feedback. When an integration fails, the team is informed immediately. This feedback loop is essential because it allows teams to respond while the context of the change is still fresh in their minds. Continuous Integration therefore shortens the time between introducing a change and discovering its impact.

Phase – Continuous Testing

Continuous Testing–

- *Some teams conduct the continuous testing phase prior to integration, whereas others conduct it after integration. In the event of a bug or error, the code is returned to the integration phase for correction. automation testing minimises the time and effort required to get reliable findings. During this stage, teams use technologies like as Selenium, BrowserStack etc*
- The continuous Integration success tells us that components fit together syntactically. It does not tell us whether the system meets user expectations, handles edge cases, or remains reliable as complexity. In other words, CI gives the team confidence that individual contributions can coexist without immediately breaking the system. However, system that builds correctly is not necessarily a system that behaves correctly under real-world conditions.
- Continuous Testing addresses this gap by validating behaviour, not just structure. Continous because , testing is embedded throughout the development lifecycle rather than postponed. Testing in DevOps is not about proving perfection. It is about reducing uncertainty.
- CT begins by shifting perspective from "what should work" to "what could fail from the user's point of view."

1. Identifying What Can Go Wrong for the User

By the time a team reaches Continuous Testing, planning has been done thoughtfully and code is being integrated regularly. In this phase, the focus shifts from structural correctness to behavioural correctness. We ask whether the system runs and it behaves correctly across different scenarios

Phase - Continuous Testing (contd)

2. Shift-Left

Instead of waiting for a feature to be complete, teams begin validating assumptions while the feature is still being designed or partially implemented. This includes validating input constraints, business rules, and failure scenarios before the feature is fully built. Whenever code is merged, tests that validate business logic, workflows, and edge cases are executed automatically. Unit tests may validate individual pieces of logic using frameworks such as PyTest or JUnit, while end-to-end tests may simulate real user actions. New tests are added to ensure that the same mistake does not happen again.

3. Testing to full Depth

Teams do not rely on a single type of test. Instead, they distribute tests across multiple layers so that failures are detected as close to their source as possible. Some of the example can be:

- Interface Testing - In SW , different components or services interact through well-defined interfaces. Interface testing ensures that these interfaces are validated continuously so that changes in one component do not break another.
- Data-driven Testing - Under this tests are executed against a wide variety of realistic data sets rather than a single happy-path input.
- Environment Testing - SW behavior can vary across environments due to configuration differences, resource limits, or integration points.

Phase – Continuous Deployment

Continuous Deployment –

- *This is the most important and active step of the DevOps lifecycle, during which the finished code is released to production servers in an automated and repeatable way. This is done with the help of many deployment tools like Kubernet , Terraform etc. The tools usage ensure consistency throughout the development, testing, production, and staging environments.*
- The system has been carefully planned, changes are integrated frequently, and behavior has been continuously tested i.e the software is technically ready. However, being "ready" does not automatically mean it is delivered. This gap between readiness and real usage is exactly why Continuous Deployment exists.
- In a DevOps environment, deployment is no longer treated as a special event. It becomes a routine activity that happens frequently and consistently. It is achieved first through automation of the deployment process. Instead of relying on manual steps that vary from person to person, deployment steps are encoded into automated workflows.

1. Identifying What "Safe Delivery" Means for System

Deployment thinking starts by identifying what failure would look like in production. Safe delivery means different things for different systems. This should mean basically , cost of failure to user not the ease of deployment to developer. Teams must assume that deployments can fail and plan recovery before failure occurs. This includes defining rollback points and understanding what state must be preserved from user perspective.

Phase - Continuous Deployment (contd)

2. Automating Deployment System

These workflows ensure that every deployment follows the same sequence of actions, same predictable path reducing human error. In practice, teams use CI/CD platforms such as Jenkins, GitHub Actions, or GitLab CI to execute these workflows consistently.

3. Packaging delivery/System

A other key method is standardizing how applications are packaged and delivered. Teams ensure that the same artifact that was tested is the one that gets deployed. This is often achieved using containers, such as Docker images, which bundle the application with its dependencies. By standardizing packaging, teams avoid the classic problem of "it worked in testing but failed in production."

Phase – Continuous Feedback

Continuous Feedback –

- *Constant feedback is implemented to assess and enhance the development phase. During this phase, client behaviour is examined for each release in an effort to enhance future releases and deployments. Companies can collect feedback using either a structured or unstructured ways.*

- After Continuous Deployment, software has moved from a controlled environment into real use. The system moves from assumptions, simulations, and controlled validations to environment where it is put to use in unpredictable ways, real workloads stress it differently, and real constraints surface that no test environment can fully replicate.
- This is precisely why Continuous Feedback come immediately after Continuous Deployment. Continuous Deployment answers the question: Can we deliver changes safely and repeatedly? Continuous Feedback answers the next, more important question: Did the delivered change actually help, harm, or confuse the user?

1. Identifying Who Can Give Meaningful Feedback

The first step in Continuous Feedback is recognizing that not all feedback sources are equal, and not all users experience the system in the same way. Teams must consciously identify who can provide meaningful insight and from which perspective.

The next challenge can be interpretation. Not every complaint points to a real problem, and not every silence means satisfaction. Continuous Feedback requires teams to distinguish signals from noise.

Phase - Continuous Feedback (contd)

2. Deciding what to change next, and why

Continuous Feedback is not about immediate action, it is about understanding the root cause. Continuous Feedback should capture whether the system is actually useful. We need to convert/listen to these signals and turn them into informed decisions. The insights should be translated into ordering priorities, refinements of existing features, or removal of unnecessary functionality. The final step in Continuous Feedback is closing the loop not just internally, but visibly. Users should be able to see that their feedback led to change. This builds trust and encourages further engagement.

Structured Feedback :

- Structured feedback refers to feedback that is collected in a predefined format, using specific questions, metrics (success /failure rates, response times etc)
- It allows teams to detect trends rather than isolated opinions.
- Tools such as monitoring dashboards, analytics platforms, issue trackers, and feedback forms are commonly used to collect data.

Un-Structured Feedback :

- Unstructured feedback refers to feedback that is not predefined, not quantified and does not fit neatly enough in to metrics. This includes feedbacks in form of free-text comments, conversations, complaints, suggestions, and observed behavior.
- It is mostly collected as user emails, chat messages, informal

Phase – Continuous Monitoring

Continuous Monitoring –

- *During this phase, the functioning and features of the application are regularly monitored to detect system faults such as low memory, security vulnerabilities or a non-reachable server etc. This procedure enables the IT staff to swiftly detect product performance issues and their underlying causes.*
- We are well aware now that modern software changes frequently. The continuous deployment assure us that the changes will reach users without causing disruption, data loss, or downtime. The core fear that we addressed was "Will our change actually behave as expected in the real world?" . In other words Continuous Deployment was about *changing the system safely*.
- Continuous Monitoring is about *understanding the system after it has changed*. Its is about *situational awareness* for eg like we will try to understand in this phase aspects like
  - Is the system healthy right now?
  - Is performance degrading over time?

Aspect	Continuous Deployment	Continuous Monitoring
Purpose	Deliver changes safely	Observe system behaviour
Focus	Change execution	System health & behaviour
Timing	During release	After release, on-going
Risk	Deployment failure	Undetected degradation

Phase - Continuous Monitoring(contd)

1. Identifying What "Healthy Behaviour" Means for the System

Monitoring must start with a definition of what normal looks like from a user' s point of view. It means choosing signals that reflect real usage/ real behaviour. That is we need to identify monitoring meaningful indicators about the system.

2. Monitoring for Degradation / Abnormal Patterns

Problems often emerge gradually rather than catastrophically. Monitoring must therefore focus on trends over time, not just binary up/down states. One example is that the system continues to function correctly, but performance is slow. No obvious errors appear in logs for such issue. There can be Functional degradation as well for eg , notifications stop reaching users , background job failure. These may be no explicit errors, only behaviour change which only monitoring of outcomes can detect over a period of time.

3. Correlating to Understand the Cause

Continuous Monitoring becomes powerful only when teams correlate multiple signals to understand cause and effect. A single metric rarely can tell the cause of the problem. For example, increased latency could mean

- ✓ higher load,
- ✓ slower dependencies,
- ✓ inefficient code, or

Phase – Continuous Operation

Continuous Operation –

- *The final phase of the DevOps lifecycle is essential for minimising scheduled maintenance and other planned downtime. Typically, developers are forced to take the server offline in order to perform updates, which increases the downtime and could cost the organisation a large amount of money.*
- This is the phase where DevOps finally becomes *day-to-day reality*, not a delivery exercise. Continuous Operations has to still exists because software systems do not fail only when changes are introduced. They fail during normal usage, under unexpected conditions, and due to factors completely unrelated to code.
- This involves handling routine operational concerns such as restarts, scaling, dependency health, and configuration management and capacity management.
- Continuous Operations must coexist with Continuous Deployment. Deployment

ebrain activations, ability, trust

1. Maintaining System Availability and Reliability

The primary responsibility of continuous operation is ensuring that the system remains available and reliable throughout daily usage 24x7x365. In practice, teams rely on automation for services like restarts, health checks , configuration management etc. Tools such as container orchestrators, process managers, or cloud auto-scaling mechanisms help ensure that transient failures do not escalate into outages.

Phase - Continuous Operation (contd)

2. Handling Incidents Without Panic

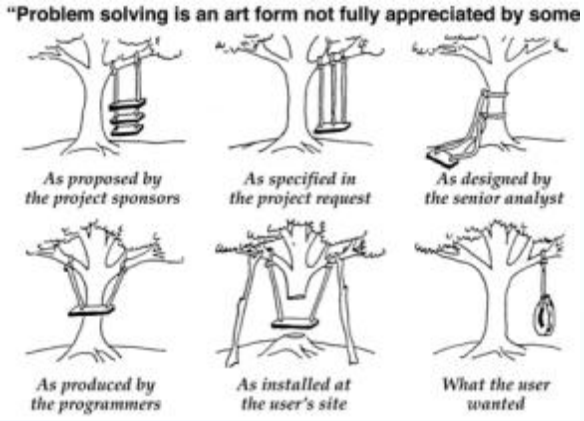
Continuous Operations does not aim to eliminate incidents entirely; it aims to handle them calmly and systematically. What this translates to is detecting incidents quickly, understanding impact, and restoring service to minimize user disruption. To achieve this clear ownership need defined , need to predefine response procedures, predefined communication and escalation channels/protocols . In simple terms what this means the continuous monitoring does not measure only absence of failure, but also focussed on quality of response.

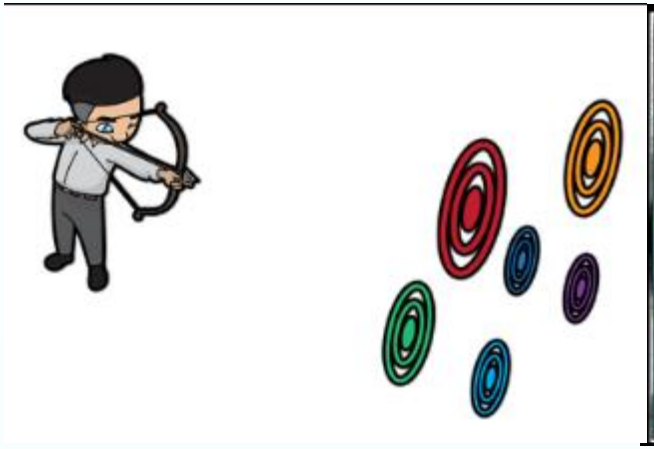
3. Using Operational Experience to Improve the System

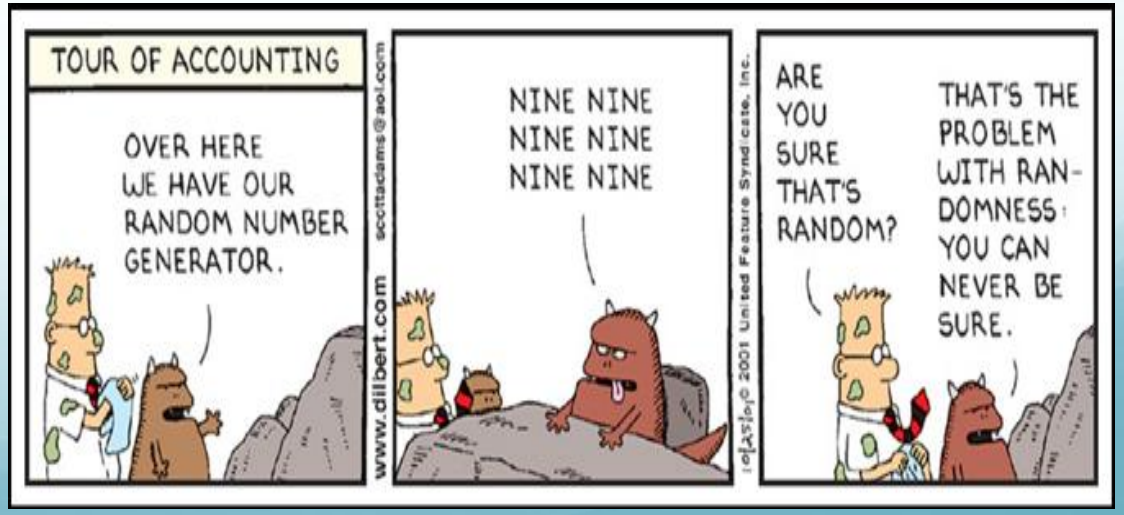
Continuous Operations is not a passive phase. Every operational issue reveals something about system design, deployment strategy, usage assumptions etc. Recurring incidents, frequent manual interventions, or operational bottlenecks signal deeper problems that must be addressed in earlier phases. Operational experience feeds directly into Continuous Feedback teams.

Benefits of DevOps

DevOps did not emerge because companies wanted faster pipelines or better tools. It emerged because traditional ways of building and running software created friction, delays, blame, and unreliable systems. The benefits of DevOps are best understood not as abstract advantages, but as direct consequences of fixing those systemic problems.







Benefits of DevOps (contd)

1. Faster and More Predictable Delivery of Software

DevOps allows teams to deliver changes frequently without fear because every phase from planning to operations reduces uncertainty & ambiguity. The speed in DevOps is a side effect, not the primary goal. The real achievement of DevOps is predictability. You shall be able to deliver changes frequently because you understand the (a) system well enough to (b) & trust each step in the delivery process.

In traditional models, delivery is slow not because teams work slowly, but because uncertainty accumulates. Integration happens late, testing happens late, feedback happens late, and deployment becomes a risky, high-stakes event.

2. Improved Software Quality and Reliability

DevOps improves quality by removing the idea that quality is someone else's responsibility or a final-stage activity. Instead, quality gets integrated and practiced throughout the lifecycle. Every phase from development, testing, deployment, monitoring, and operations contributes to system reliability & speed in DevOps.

Continuous integration ensures conflicting changes are caught early. Continuous testing validates behaviour repeatedly, not just once. Continuous monitoring exposes real-world degradation that tests cannot simulate. Continuous operations ensure that day-to-day reliability is maintained and deep bugs are sent back to development teams even when no new features are

Benefits of DevOps (contd)

3. Reduced Risk During Changes and Releases

By deploying small increments and validating them continuously, teams reduce the magnitude of failures. If something goes wrong, the impact is limited and recovery is faster. Rollbacks are planned, not improvised. Monitoring detects issues early, before users experience widespread disruption.
Traditional models treat releases as one time affair, a large events, which concentrates risk. DevOps distributes risk across many small changes, making each change easier to understand, test, and recover from.

4. Faster Detection and Resolution of Issues

DevOps reduces the time between a problem occurring and the team becoming aware of it. Continuous monitoring ensures teams see issues as they emerge, not after users complain. Clear ownership and automation allow teams to respond quickly and systematically.
In traditional environments, issues are often discovered late and investigated manually under pressure. DevOps replaces this chaos with visibility and preparedness , predefined process and protocols.

5. Better Alignment Between Development, Operations, and Users

DevOps bridges the gaps between development, operations, and other cross-functional teams by breaking down communication barriers and promoting a culture of shared responsibility. Clear focus & priorities means No to "not our problem" mentality resulting in faster velocity and good quality

Collaboration & Communication

- Collaboration and communication are foundational to DevOps …When we talk about DevOps, it is easy to think of it as a collection of tools, pipelines, and automation practices.
- … But at its core, DevOps is not a technical transformation—it is a human and organizational transformation. It’ s not about / It cannot be… tools talking to tools.
- It breaks down the gaps that inherently exists between development, operations, and security teams and tries to replace it with a
  - ✓ Shared sense of responsibility
  - ✓ Enable a faster delivery and
  - ✓ Improved quality.

Challenge (Disconnected Teams) ..

Ineffective Communication Channels ➔

Unreliable or inefficient communication tools slow information flow and increases inefficiency to build …

Silos and Misunderstandings ➔

Divides between Dev and Ops teams can create communication barriers and hinder DevOps collaboration…

Lack of Shared Goals ➔

Without clear, shared objectives, teams may prioritize different things, leading to misalignment in meeting deadline/s…

Collaboration & Communication (contd)

What collaboration means in a DevOps environment ?

- ✓ Developers think about deployment and operations while writing code.
- ✓ Operations teams influence system design early.
- ✓ Testing is integrated into development, not performed at the end.
- ✓ Quality/Meaningful/ Relevant Feedbacks reaches developers quickly from multiple directions.

Shared Objectives …

During Continuous Development, teams discuss user needs and refine objectives.
During Continuous Integration, developers receive immediate feedback on delivery.
During Continuous Deployment, teams coordinate safe release strategies.
During Continuous Monitoring, users / operation signals are shared across teams.
During Continuous Operation, operational insights influence planning decisions.

Collaboration & Communication (contd)

How collaboration and communication are achieved ?

Can rely on good intentions alone...
It has to be Practiced …

1. Cross-Functional Teams

One of the most fundamental methods is restructuring teams so that they are no longer separated strictly by function. Instead of having isolated development, testing, and operations departments, DevOps promotes **cross-functional teams** where people with different expertise work together till product is delivered.
All members share responsibility for the system’s success and not their team’s Milestones !!

2. Establishing Regular Communication Channels

One of the most popular practice is Daily Stand-Up Meetings which are short, focused meetings (typically 15 minutes) for team members to share progress updates, roadblocks, and dependencies. Another method is to provide regular project updates and maintain up-to-date documentation to keep everyone informed. Frequent Retrospective meetings reflects on how we worked, not just what we delivered. Very powerful mode of communication when done honestly .. No Blame Games here.

Collaboration & Communication (contd)

3. Transparent Work Tracking Systems

Instead of information being scattered across emails, personal notes, or private conversations a transparent work tracking system is one best method to increase collaboration and communication where all team members can see what work is planned, what is in progress, what is blocked, and what has been completed. It acts as a single, visible source of truth for the entire project.

4. Automation and Communication Tools

Automation tools are built into the workflows that make information visible, automate feedback, and connect teams working in different stages of the software lifecycle. Tools , if adopted , it helps in way that rather than asking people to constantly report status, the system itself communicates what is happening. Some eg are :

- Version Control Systems (VCS) like Git
- Chat Applications
- Project Management Tools (Jira, BugJilla etc ..)
- Monitoring Dashboards

5. Team-Building Activities / Recognizing and Rewarding

Companies invest in team-building activities to fortify bonds, enhance communication, and cultivate a spirit of unity. These activities can range from informal social events to structured exercises promoting teamwork.

Customer Centric Decision Making

- After Collaboration & Communication, the next logical concept in DevOps is Customer-Centric Decision Making, because once teams collaborate effectively, the next question becomes:
  - Who are we collaborating for, and what drives our decisions?
- DevOps teams is to deliver what customers truly need and desire, aligning development with user experience and expectations. It’s a simple concept that can be complex in execution. So a truly customer-centric approach may also means consistent recalibration based on customer-focused considerations.

What customer-centric decision making means

Is this possible ??

- ✓ Instead of optimizing for internal convenience, team preferences, or technical elegance , what Customer-centric decision making means is that every technical and process decision is guided by its impact on the end user and not meeting project requirements only.
- ✓ In traditional software environments, progress is often measured in terms of features delivered. In this model, the focus is on output—the amount of functionality produced. The goal is completing everything on the backlog list, regardless of whether those features actually improve the user’s experience or not. Customer-centric decision making forces a value-driven development approach that shifts the focus from what is built to what

Customer Centric Decision Making (contd)

1. Continuous User Feedback Loops

One of the most direct ways to remain customer-centric is to establish regular and continuous channels for user feedback. Instead of waiting until the end of a project, teams can collect user input throughout the lifecycle. This feedback can come from any methods : Structured or Unstructured. The key idea is that decisions should be based on real user experiences, not assumptions.

2. Prioritizing Work Based on User Value

Customer-centric teams do not treat all tasks equally. They prioritize work based on the value delivered to the user, not on technical interest or internal convenience. This requires asking questions such as Which issue causes the most frustration? Which improvement saves users the most time? Which feature is most frequently requested?

3. Cross-Functional Decision Making with User Context

Customer-centric decisions improve when multiple perspectives are involved. Developers, testers, operations staff, and product owners all see different aspects of the system. The improvement is because Cross-functional discussions help answer questions like Will this feature create operational issues? Will this change increase response time? Will users understand this interface?

Customer Centric Decision Making (contd)

4. Frequent Small Releases to Validate Assumptions

Customer-centric teams avoid large, infrequent releases. Instead, they release small improvements regularly with delivery in increments and observe how users respond. This allows teams to test ideas in real environments , gather feedback and correct system issue quickly , avoid building unnecessary features

5. Monitoring Real Usage and Behaviour

Users often behave differently than expected. What teams think is important may not be what users actually use. Therefore, customer-centric teams rely on usage data, not just opinions. So this involves monitoring users in his environment very closely and carefully. We can try and understand which features are used frequently ? where users abandon tasks ? response times , error rates etc.

Assignment-1

Assignment Description:

Now when we have completed all the phases of DevOps you need to justify as your first assignment (in not more than 2 A4 pages) on should we OR should we not apply DevOps in startup project that is entrusted to prepare a small IT solution for any of the problem listed below.

Assignment Submission Deadline :

AIML ➔ 26th Feb

IT1/IT2 ➔ 27th Feb

Sr#	Project Name
1	Smart Attendance System
2	Campus Event Management System
3	Hostel Management System
4	Department Resources Booking System
5	Placement Management System
6	Lost & Found Management System

Thanks